Individual Student Contribution Summary — Student 3

Student Name: Ahad Gour
Institution: MIT Academy of Engineering (MITAOE)
Project Title: NetSage AI: Automated Cisco Packet Tracer Diagnostic & Troubleshooting Framework
Assigned Project Role: Python Automation Developer & Responsible AI Audit Lead

---

1. Executive Summary of Contributions

My primary responsibility was developing the Python rule checker engine (`rule_checker.py`), conducting human-in-the-loop audit evaluations of AI predictions (`responsible_ai_log.csv`), building the interactive analytics dashboard (`generate_dashboard.py` & `dashboard/`), and authoring the main project report (`report.md`).

---

2. Detailed Task Breakdown & Deliverables Owned

Task 1: Deterministic Python Rule Checker Engine (`rule_checker.py`)

- Developed the standalone Python rule checker script `rule_checker.py` .
- Programmed object-oriented checking functions inside `NetworkRuleChecker` :
  - `check_duplicate_ips()` — Detects duplicate IP address assignments across routers and hosts.
  - `check_subnet_mask_mismatches()` — Detects subnet mask conflicts on shared broadcast domains.
  - `check_default_gateway_mismatches()` — Flags mismatch between host gateway IP and router interface IP.
  - `check_admin_down_interfaces()` — Identifies configured interfaces marked as `"administratively down"` .
  - `check_missing_vlans()` — Catches access ports configured with VLAN IDs missing from the switch database.

    - `check_missing_routes()` — Verifies router routing tables for missing target subnets or default routes.
- Executed verification tests and generated `rule_checker_output.txt`.

Task 2: Responsible AI Human Audit & Review (`responsible_ai_log.csv`)

- Created the Responsible AI audit log `responsible_ai_log.csv`.
- Audited all 21 AI diagnostic responses against confirmed ground truth, recording human reviewer decisions:
    - **Accepted (16 Cases - 76.2%):** Verified AI accuracy for standard single-fault scenarios.
    - **Edited (3 Cases - 14.3%):** Applied human engineering corrections (e.g. adding `no access-list 1` cleanup in `CASE-014`; adding interface reset `shutdown` / `no shutdown` pair in `CASE-016`).
    - **Rejected (2 Cases - 9.5%):** Documented error root causes for AI hallucinations:
        - *`CASE-019` (OSPF Router ID Collision):* AI incorrectly claimed physical cable flapping; human corrected to OSPF Router ID conflict (`1.1.1.1`).
        - *`CASE-020` (Subinterface Dot1Q Mismatch):* AI misread subinterface `.20` as VLAN tag 10; human corrected to `encapsulation dot1Q 20`.

Task 3: Interactive Dashboard & Metrics Engine (`generate_dashboard.py` & `dashboard/`)

- Developed `generate_dashboard.py` to automatically compute project metrics and build visual reports.
- Generated output files in `dashboard/`:
    - `dashboard/dashboard.html` — Interactive visual web dashboard with summary statistics and charts.
    - `dashboard/dashboard.xlsx` — Formatted Excel workbook containing raw case metrics.
    - `dashboard/dashboard_chart.png` — Static chart graphic displaying decision distribution.
    - `dashboard/dashboard_metrics.csv` — CSV summary of key performance indicators.

Task 4: Official Project Documentation (`report.md`)

- Authored the official comprehensive project report `report.md` written in clear, simple English covering project background, topology specs, rule checking logic, AI audit metrics, and usage guides.

---

3. Primary Owned File Deliverables

1. `rule_checker.py` & `rule_checker_output.txt` — Deterministic Python rule checker engine and output log.
2. `responsible_ai_log.csv` — Responsible AI audit log tracking human agreement, edits, and rejections.
3. `generate_dashboard.py` & `dashboard/` — Automated dashboard generator script and HTML/Excel dashboard files.
4. `report.md` — Official final project report.

---

4. Verification & Testing

- Executed `rule_checker.py` and `generate_dashboard.py` via command line to verify 100% clean execution with exit code 0.
- Verified that all visual charts, metrics, and audit tables rendered accurately in `dashboard.html`.

---

5. Summary of Impact

My contributions provided **automation, quality assurance, and visual reporting** for NetSage AI. The Python rule checker provided instant deterministic fault verification, while the Responsible AI audit guaranteed safety and accountability for AI-generated network advice.